OOPs Concepts in Java

06 October 2026 20:19

Object - Oriented Programming is a paradigm that provides concepts such as inheritance, data binding, polymorphism, etc.

Simula is considered the first object - oriented programming language. The programming paradigm where everything is represented as an object is known as pure Object Oriented Programming Language.

Smalltalk is considered the first truly object - oriented programming language.

The popular object - oriented languages are Java, Javascript, C #, PHP, Python, C++, etc.

OOPs(Object - Oriented Programming)

Object - Oriented Programming aims to implement real - world entities. For example objects, classes, abstraction, inheritance, polymorphism, etc.

Object means the real-world entities such as pen, chair, table, computer, watch, etc. **Object-oriented Programming** is a methodology or paradigm for designing a program using classes and objects. It simplifies software development and maintenance by providing some concepts:

- **Object**
- **Class**
- **Inheritance**
- **Polymorphism**
- **Abstraction**
- **Encapsulation**

Apart from these concepts, there are some other terms which are used in Object - Oriented Design:

- ***Coupling***
- ***Cohesion***
- ***Association***
- ***Aggregation***
- ***Composition***

Object

Any entity has **state** and **behavior** is known as object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.

An **Object** can be defined as an **instance** of a class. It contains an address and takes up some space and memory. Objects can communicate without knowing the details of each other's data or code. The only necessary thing is the type of message accepted and the type of response returned by the objects.

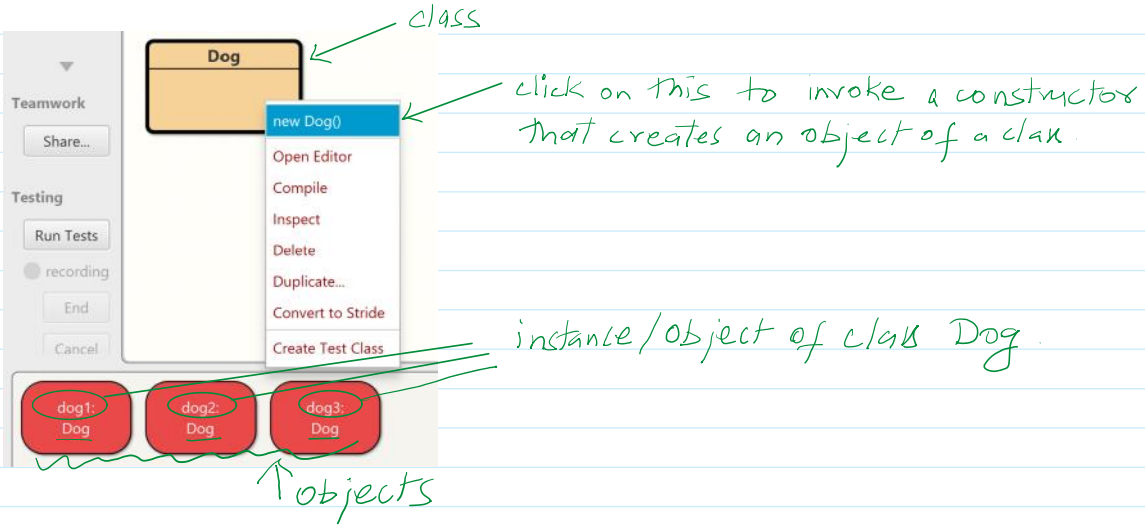
Example: A dog is an object because it has states like color, name, breed, cetera as well as behaviors like wagging the tail, barking,

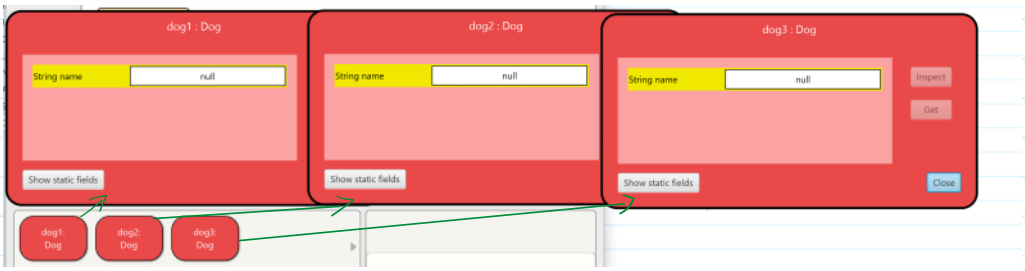
eating, etc.

The primary element of an object are:

- **State:** It is embodied in an object's properties. Additionally it reflects and objects attribute.
- **Behavior:** An object's methods are representation of it. Additionally, it represents how an object reacts to other object.
- **Identity:** Identity is a special name that gives a thing the ability to communicate with other objects.
- **Method:** A method is a group of statements that carry out a certain operation and gives an outcome. A method can complete a certain task without giving back any results. Methods are regarded as time saver since they enable us to reuse the code without having to type it again. Java differs from languages like C, C++ and Python in that each method needs to be a component of a class.

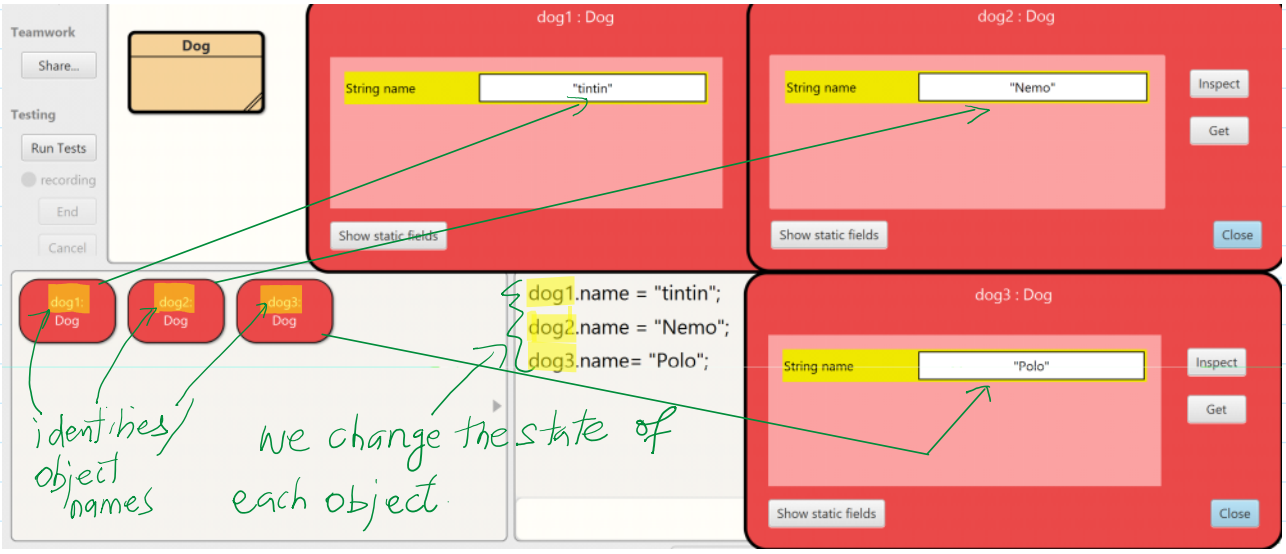
Example of Object:
The following example demonstrates how to create a class. Instance an object, assign values to object fields and invoke method using that object.

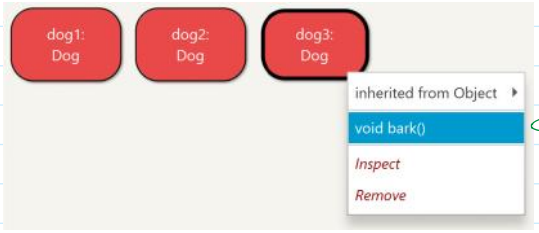




Every object has its own attribut called name.

To access these attributes we have to use object_name.variable_name

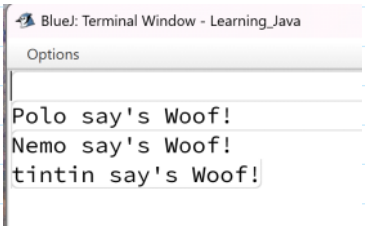


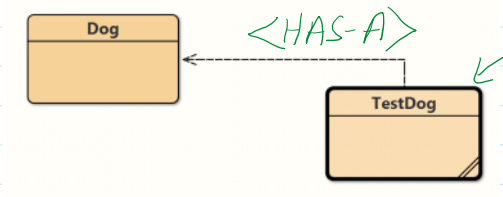


← Common behavior of each object

↑ Methods

↑ is called using object, the method uses objects attribute(s), data to execute its code





← This class contains the Objects of Dog class

TestDog —<HAS-A>→ Dog

```
package ClassAndObject;


/**
 * Write a description of class Dog here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Dog
{
    String name; //property or attribute or state
```

access modifier

Java keyword

class name

```
{
    String name; //property or attribute or state

    void bark(){ //behavior
        System.out.print("\n"+name + " say's Woof!");
    }
}
```

class name

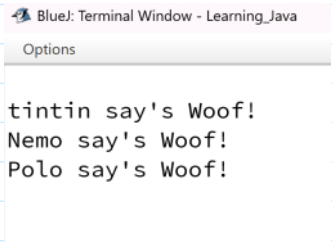
Code reused

```
package ClassAndObject;

/**
 * Write a description of class TestDog here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestDog
{
    public static void main(String[] args){
        System.out.print("\f");
        Dog dog1 = new Dog(); //creating objects
        Dog dog2 = new Dog();
        Dog dog3 = new Dog();
        dog1.name = "tintin"; // Assigning values to the objects field.
        dog2.name = "Nemo";
        dog3.name= "Polo";

        //calling method on the object(s)
        dog1.bark();
        dog2.bark();
        dog3.bark();
    }
}
```

Output:



Class

Class is a logical entity. A class can also be defined as a blueprint from which we can create an individual object. Class does not consume any space.

In general, a class declaration includes the following in the order as it appears:

1. Modifiers: A class can be public or have default access.

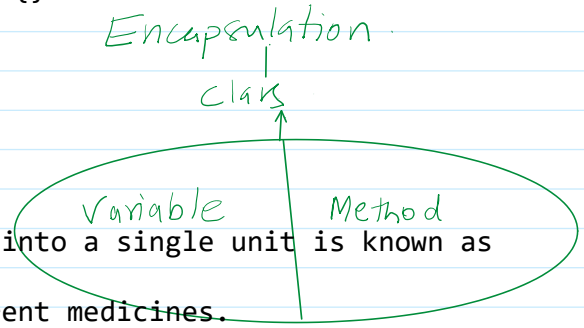
2. **class keyword:** The class keyword is used to create a class.

3. **Class name:** To begin with an initial letter (capitalized by convention)

4. **Superclass(if any):** The name of the class's parent (super class) if any preceded by the keyword **extends**. A class can only extend (subclass) 1 parent.

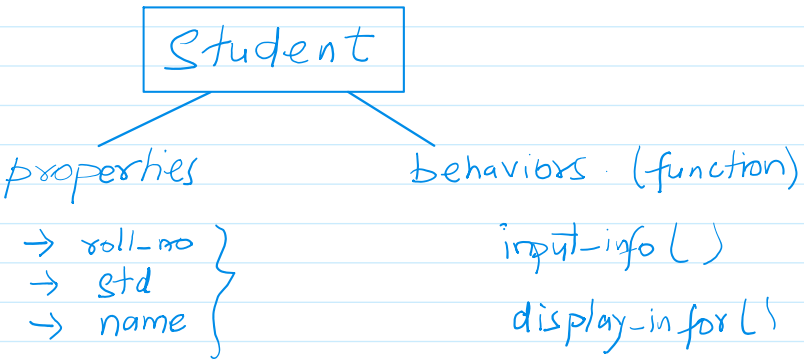
5. **Interface(if any):** A comma - separated list of interfaces implemented by the class, if any preceded by the keywords **implements**. A class can implement more than one interface.

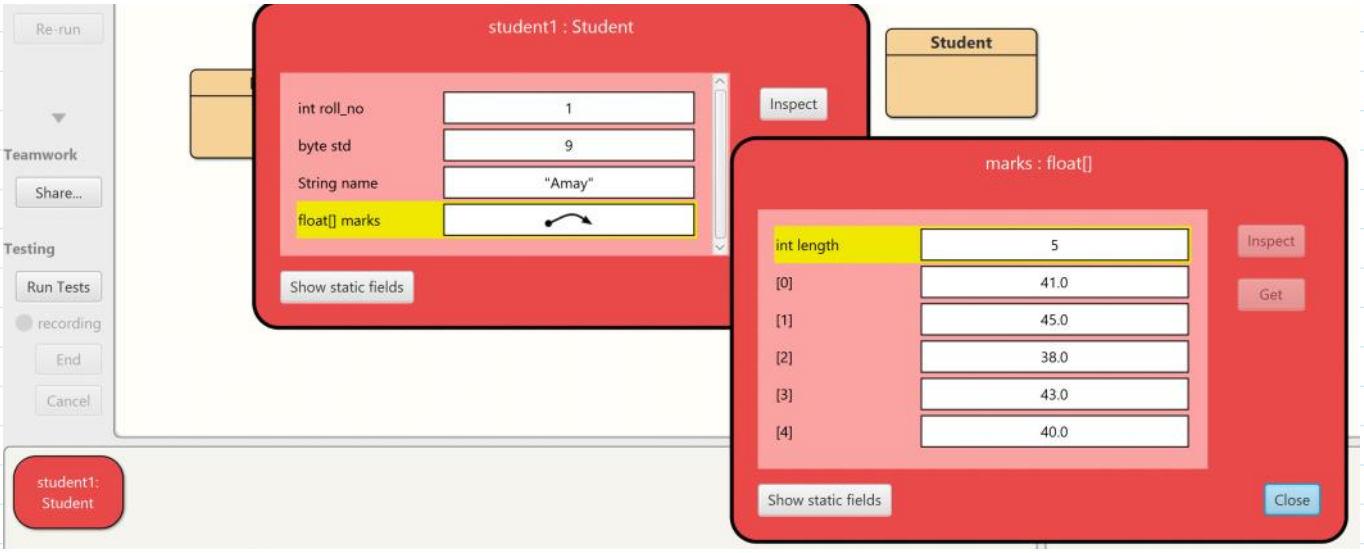
6. **Body**: The class body surrounded by braces, {}.



Encapsulation
Binding (or wrapping) Code and data together into a single unit is known as **encapsulation.**
For example, a capsule is wrapped with different medicines.

A Java class is an example of encapsulation. A Java Bean is a fully encapsulated class because all the data members are private.






```
package ClassAndObject;
import java.util.*;


/**
 * Write a description of class Student here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student
{
    int roll_no;
    byte std;
    String name;
    float marks[];
    static int numOfSubjects = 5;
    static Scanner sc = new Scanner(System.in);
    void insert_info(){
        System.out.print("\nEnter roll number: ");
        roll_no = sc.nextInt();
        System.out.print("\nEnter standard: ");
        std = sc.nextByte();
        sc.nextLine(); //to remove keyboard buffer.
        System.out.print("\nEnter name: ");
        name = sc.nextLine();
        marks = new float[numOfSubjects];
        int i;
        String subjects[] = {"English", "Math", "Kanada", "Science",
"Humanities"};
        for(i = 0; i < numOfSubjects; i++){
            System.out.print("\nEnter the marks of " + subjects[i]+": ");
            marks[i] = sc.nextFloat();
        }
    }

    void display_info(){
```

```
        System.out.print("\n============================");
        System.out.print("\n===+Student Information+===");
        System.out.print("\n============================");
        System.out.print("\nRoll number: " + roll_no +
        "\nName: " + name +
        "\nStandard: " + std +
        "\nMarks: ");
        int i ;
        for(i = 0; i < numOfSubjects - 1; i++){
            System.out.print(marks[i]+ ", ");
        }
        System.out.println(marks[i]); //print the marks first and then change the
line
    }
}

package ClassAndObject;


/**
 * Write a description of class DensAcademy here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class DensAcademy
{
    public static void main(String[] args){
        System.out.print("\f");
        Student st1 = new Student();
        Student st2 = new Student();
        Student st3 = new Student();
        // Student st4 = new Student();
        // Student st5 = new Student();
        // Student st6 = new Student();

        st1.insert_info();
        st2.insert_info();
        st3.insert_info();
        // st4.insert_info();
        // st5.insert_info();
        // st6.insert_info();

        st1.display_info();
        st2.display_info();
        st3.display_info();
        // st4.display_info();
        // st5.display_info();
        // st6.display_info();
    }
}

Output:

   Enter roll number: 1
```

```
Enter standard: 9

Enter name: Jiya

Enter the marks of English: 45.75

Enter the marks of Math: 49

Enter the marks of Kanada: 41

Enter the marks of Science: 45.5

Enter the marks of Humanities: 44.5

Enter roll number: 2

Enter standard: 9

Enter name: Aarav

Enter the marks of English: 41

Enter the marks of Math: 40

Enter the marks of Kanada: 5

Enter the marks of Science: 41

Enter the marks of Humanities: 38

Enter roll number: 3

Enter standard: 9

Enter name: Aadya

Enter the marks of English: 48

Enter the marks of Math: 50

Enter the marks of Kanada: 41

Enter the marks of Science: 48

Enter the marks of Humanities: 45

==========================
===+Student Information+===
==========================
Roll number: 1
Name: Jiya
Standard: 9
Marks: 45.75, 49.0, 41.0, 45.5, 44.5
```

```
==========================
===+Student Information+===
==========================
Roll number: 2
Name: Aarav
Standard: 9
Marks: 41.0, 40.0, 5.0, 41.0, 38.0

==========================
===+Student Information+===
==========================
Roll number: 3
Name: Aadya
Standard: 9
Marks: 48.0, 50.0, 41.0, 48.0, 45.0
```

Difference Between class and object in Java

Feature	Class	Object
Definition	A class is a blueprint or template for creating objects.	It is an instance of a class.
Type	It is a logical entity.	It is a real world entity.
Memory allocation	Class definition does not occupy memory.	It occupies memory when an object is created.
Keyword	It is created by using "class" keyword.	It is created using the 'new' keyword and a class constructor.
Contains	It contains properties and behavior that an object will have.	It is used to access the properties and behavior defined in the class.
Existence	Exist only in source code.	Exist in runtime (in memory).
Usage	Defines the structure and behavior of objects.	Represents an entity with actual values.
Syntax	class ClassName{…}	ClassName objectName = new ClassName();
Life	It exists until the program runs.	It exists as long as the object is referenced in the program.
Inheritance	The classes can be extended using inheritance.	It cannot be inherited because objects are specific instances.